

Reviewer Pack — Newton Polytope Volume Inverse Proportional to SOS Degree fo...

Entry 9dba9c55f385 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 08:32:10 UTC

Newton Polytope Volume Inverse Proportional to SOS Degree for Max-CUT

Entry ID: 9dba9c55f385 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 08:32:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Newton Polytopes) **Field B** (complexity object): SOS Degree for Max-CUT Instances

Statement:

For a Max-CUT instance with n variables, the volume of the Newton polytope of its corresponding degree- d SOS relaxation polynomial is $\Theta(1/d^2)$. The volume is inversely proportional to d^2 , with the constant depending on the instance's edge density.

Rationale (proposer's reasoning):

Newton polytopes encode the monomial structure of polynomials, which directly reflects the constraint geometry of SOS relaxations. The volume captures the 'spread' of monomials, which may limit the efficiency of higher-degree relaxations by increasing the number of terms in the Gram matrix.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eb8262c739dd68cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    def map_clauses_to_monomials(edges, d):
        monomials = set()
        for edge in edges:
            for clause in combinations(range(len(edge)), d):
                monomial = tuple(sorted(clause))
                monomials.add(monomial)
        return monomials

    def compute_newton_polytope_volume(monomials):
        n = len(monomials)
        vertices = []
        for monomial in monomials:
            vertex = [0] * (n + 1)
            for i, m in enumerate(monomial):
                vertex[m] += 1
            vertices.append(vertex)

    def determinant(matrix):
        if len(matrix) == 2:
            return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
        det = 0
        for j in range(len(matrix)):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
```

```

        return det

    return abs(determinant(vertices))

def compute_sos_degree(monomials):
    degree = 0
    for monomial in monomials:
        degree = max(degree, sum(monomial))
    return degree

n = random.randint(5, 40)
edges = generate_max_cut_instance(n)
d_values = [2, 3, 4, 5, 6, 7, 8, 9, 10]

volumes = []
for d in d_values:
    monomials = map_clauses_to_monomials(edges, d)
    volume = compute_newton_polytope_volume(monomials)
    volumes.append(volume)

if len(volumes) < len(d_values):
    return {
        "metric_name": "Volume",
        "metric_value": None,
        "instances_tested": len(volumes),
        "conjecture_holds": False,
        "counterexample": "Insufficient data"
    }

d_values = [2, 3, 4, 5, 6, 7, 8, 9, 10]
expected_volumes = [volumes[0] / (d ** 2) for d in d_values]

supports_conjecture = all(abs(v - e) < 1e-6 for v, e in zip(volumes, expected_volumes))

return {
    "metric_name": "Volume",
    "metric_value": volumes[0],
    "instances_tested": len(volumes),
    "conjecture_holds": supports_conjecture,
    "counterexample": "" if supports_conjecture else f"Volumes: {volumes}, Expected: {expected_volumes}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_volume = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_volume = math.sqrt(sum((r["metric_value"] - mean_volume) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_volume} std={std_volume} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
TRIAL: {'metric_name': 'Volume', 'metric_value': 0, 'instances_tested': 9, 'conjecture_holds': True,
RESULT: SUPPORTED mean=0.0 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n too small: 9 instances tested likely correspond to $n \leq 15$, where volume scales trivially with n . Metric saturation: all metric values are 0, making the bound vacuous. Selection bias: instances may avoid adversarial cases where volume deviates from 0.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Tested instances show perfect support but suffer from $n \leq 15$ (trivial scaling), metric saturation at 0, and potential selection bias. Pre-registered criteria require $\geq 80\%$ seeds, which is met, but critic challenges invalidate confidence. | next: Test with larger n (≥ 30) and adversarial edge density configurations to probe non-trivial volume scaling behavior

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	88591
2	preregistration	ollama_remote	qwen3:8b	0	0	24515
3	novelty	ollama_remote	qwen3:8b	0	0	20683
4	novelty	ollama_remote	qwen3:8b	0	0	13439
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22857
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11829
7	critic	ollama_remote	qwen3:8b	0	0	30069
8	judge	ollama_remote	qwen3:8b	0	0	15903

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 227885 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/9dba9c55f385.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9dba9c55f385.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9dba9c55f385.tar.gz` (if generated)